

LLM Council v3.0 - Comprehensive Project Report

1. Project Overview & Architecture

LLM Council v3.0 is a fully offline, multi-agent AI deliberation system designed to provide high-quality reasoning on complex queries. It simulates a panel of 4 distinct AI agents (Philosopher, Logician, Visionary, Challenger) and 1 Judge. This architecture provides robust, multi-faceted analysis without relying on any external APIs.

Architecture Stack:

- Frontend: Next.js (App Router), React, Tailwind CSS, Framer Motion for JARVIS-style holographic UI.
- ML Inference Engine: Asynchronous Python microservice (aiohttp) paired with llama-cpp-python.
- Containerization: Fully Dockerized via docker-compose with seamless internal networking.

2. User Interface & Advanced Features

The frontend is designed with premium 'glassmorphism' aesthetics and includes several advanced features:

- Voice Input: Integrated with the Web Speech API (webkitSpeechRecognition) for seamless verbal query input.
- Text-to-Speech (TTS): Synthesizes the Judge's final verdict using the browser's native SpeechSynthesis API.
- Real-Time Inference: Utilizes Server-Sent Events (SSE) to stream multiple LLM outputs concurrently to the browser.
- Exporting: Allows the user to instantly download the complete council debate as JSON or Markdown.

3. Security & Privacy Levels

Security is absolute in v3.0:

- Strict Headers: The Next.js application enforces Content-Security-Policy (CSP), X-Frame-Options (SAMEORIGIN), and Strict-Transport-Security to defend against XSS, clickjacking, and MITM attacks.
- Offline Isolation: 100% of inference is executed locally using quantized GGUF models. Zero telemetry or data is sent to external APIs.
- API Protection: The ML Engine employs strict CORS policies restricting access only to the frontend container, plus memory-based rate limiting to prevent abuse.
- Dependency Risk: All dependencies are locked, and outdated legacy API wrappers have been removed.

4. Council Members & ML Models

LLM Council v3.0 - Comprehensive Project Report

The project leverages highly-optimized 1B to 3B parameter models designed for local CPU/GPU execution:

1. Sage (Philosopher) -> Llama-3.2-1B-Instruct-Q4
2. Analyst (Logician) -> Qwen2.5-1.5B-Instruct-Q4
3. Strategist (Visionary) -> Phi-3-mini-4k-instruct-Q4
4. Skeptic (Challenger) -> Gemma-2-2B-it-Q4
5. Judge (Head) -> Llama-3.2-3B-Instruct-Q4

5. Deployment Guidelines

The project is fully pre-configured for production deployment via Docker Compose:

- Execute ``docker-compose up --build -d`` to launch both the frontend and ML engine containers.
- The Next.js frontend will be exposed on port 3000, connecting internally to the ML engine on port 8001.
- Hardware Requirements: Minimum 16GB RAM for concurrent inference of the 5 quantized models, with optional NVIDIA GPU acceleration config.